

ABSTRACT

In this project we propose and implement a general convolutional neural network (CNN) building framework for designing real-time CNNs. We validate our models by creating a real-time vision system which accomplishes the tasks of face detection, gender classification and emotion classification simultaneously in one blended step using our proposed CNN architecture. After presenting the details of the training procedure setup we proceed to evaluate on standard benchmark sets. Along with this we also introduced the very recent real-time enabled guided back-propagation visualization technique. We argue that the careful implementation of modern CNN architectures, the use of the current regularization methods and the visualization of previously hidden features are necessary in order to reduce the gap between slow performances and real-time architectures.

TABLE OF CONTENTS

Contents

1	Introduction	4
2	Problem Statement	5
3	Objective	5
4	Literature Review	6
5	Methodology	7
6	Implementation	12
6.1	Pseudocode and Functions	12
7	Result	16
8	Conclusion and Future Scope	18
9	Software and Hardware requirements	19
10	Schedule	19
11	References	20

INTRODUCTION

The success of service systems decisively depends on a smooth system to user interaction. Thus, a system should be able to extract information just from the face of its user, e.g. identify the emotional state or deduce gender. Interpreting correctly any of these elements using machine learning (ML) techniques has proven to be complicated due the high variability of the samples within each task. This leads to models with millions of parameters trained under thousands of samples. Furthermore, the human accuracy for classifying an image of a face in one of 7 different emotions is $65\% \pm 5\%$. One can observe the difficulty of this task by trying to manually classify the FER-2013 dataset images in Figure1 within the following classes { “angry”, “disgust”, “fear”, “happy”, “sad”, “surprise”, “neutral” }. In spite of these difficulties, such system platforms oriented to attend and solve household tasks require facial expressions systems that are robust and computationally efficient. Moreover, the state-of-the-art methods in image-related tasks such as image classification [1] and object detection are all based on Convolutional Neural Networks (CNNs). These tasks require CNN architectures with millions of parameters; therefore, their deployment in such platforms and real-time systems becomes unfeasible. In this project we propose and implement a general CNN building framework for designing real-time CNNs. The implementations will be validated in a real-time facial expression system that provides face-detection, gender classification and that achieves human-level performance when classifying emotions. Furthermore, CNNs are used as black-boxes and often their learned features remain hidden, making it complicated to establish a balance between their classification accuracy and unnecessary parameters. Therefore, we will implement a real-time visualization of the guided-gradient back-propagation proposed by Springenberg [8] in order to validate the features learned by the CNN.

PROBLEM STATEMENT

A system should be able to extract information just from the face of its user, e.g. identify the emotional state or deduce gender. Interpreting correctly any of these elements using machine learning (ML) techniques has proven to be complicated due the high variability of the samples within each task. This leads to models with millions of parameters trained under thousands of samples. In spite of these difficulties, systems that are oriented to attend and solve such tasks require facial expressions systems that are robust and computationally efficient. These tasks require CNN architectures with millions of parameters; therefore, their deployment in robot platforms and real-time systems becomes unfeasible. In this project we propose an implementation of a general convolutional neural network (CNN) building framework for designing real-time CNNs. We will be validating our models by creating a real-time vision system which accomplishes the tasks of face detection, gender classification and emotion classification simultaneously in one blended step using our proposed CNN architecture.

OBJECTIVE

In deep learning, a convolutional neural network (CNN, or ConvNet) is a class of deep neural networks, most commonly applied to analyzing visual imagery. CNNs are regularized versions of multilayer perceptrons. Multilayer perceptrons usually mean fully connected networks, that is, each neuron in one layer is connected to all neurons in the next layer. CNNs use relatively little pre-processing compared to other image classification algorithms. In this project, we demonstrate the advantage of this novel approach in which we will be able to do the following -:

- Implement a general CNN building framework for designing real-time CNNs. The implementations have been validated in a real-time facial expression system that provides face-detection.
- Furthermore, we will be also doing gender classification and that achieves human-level performance when classifying emotions.

LITERATURE REVIEW

Commonly used CNNs for feature extraction include a set of fully connected layers at the end. Fully connected layers tend to contain most of the parameters in a CNN. Here is the conclusion of some of the reference paper that we review to make our project better and to know more technologies that we can use in our system.

- In paper [1] by Francois Chollet. Xception: Deep learning with depth wise separable convolutions. It describes that how modern CNN architectures leverage from the combination of two of the most successful experimental assumptions in CNNs: the use of residual modules and depth-wise separable convolutions.
- In paper [2] by Andrew G. Howard et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications; it is described that Depth-wise separable convolutions reduces the computation with respect to the standard convolutions.
- In paper [4] by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition; it shows how Residual modules modify the desired mapping between two subsequent layers, so that the learned features become the difference of the original feature map and the desired features.
- In paper [7] by Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition; describes how a layer contains approximately 90% of all its parameters in their last fully connected layers.
- In paper [9] by Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision; it describes that how we can reduce the amount of parameters in their last layers by including a Global Average Pooling operation.

METHODOLOGY

Reducing the number of parameters help us overcoming two important problems. First, the use of small CNNs alleviate us from slow performances in hardware-constrained systems such robot platforms. And second, the reduction of parameters provides a better generalization. Our first model relies on the idea of eliminating completely the fully connected layers. The second architecture combines the deletion of the fully connected layer and the inclusion of the combined depth-wise separable convolutions and residual modules. Following the previous architecture schemas, our initial architecture used Global Average Pooling to completely remove any fully connected layers. This will be achieved by having in the last convolutional layer the same number of feature maps as number of classes, and applying a softmax activation function to each reduced feature map. Our initial proposed architecture is a standard fully-convolutional neural network composed of 9 convolution layers, ReLUs [3], Batch Normalization [5] and Global Average Pooling.

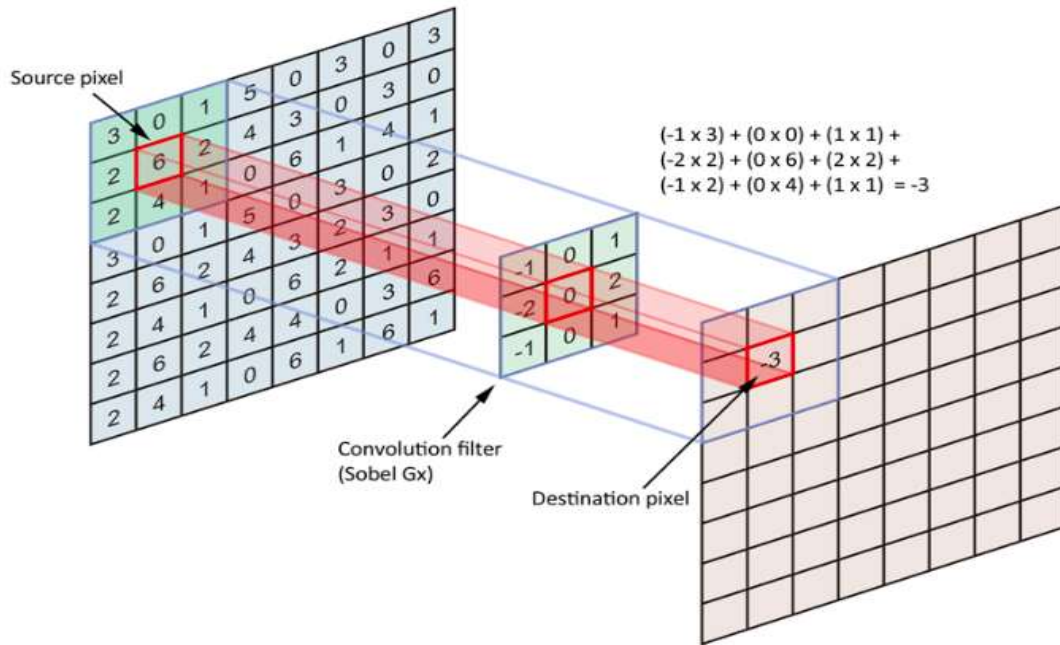


Fig. 1 A simple CNN model

Since our initial proposed architecture deleted the last fully connected layer, we reduced further the amount of parameters by eliminating them now from the convolutional layers.

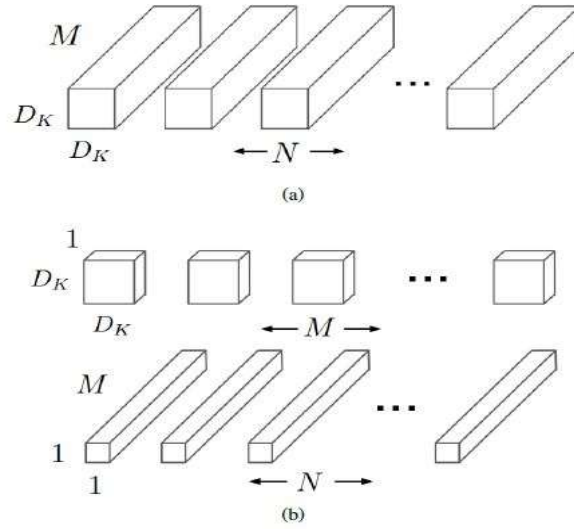


Fig. 2: Difference between (a) standard convolutions and (b) depth-wise separable convolutions.

This was done through the use of depth-wise separable convolutions. Depth-wise separable convolutions are composed of two different layers: depth-wise convolutions and pointwise convolutions. The main purpose of these layers is to separate the spatial cross-correlations from the channel cross-correlations [1]. They do this by first applying a $D * D$ filter on every M input channels and then applying $N 1 * 1 * M$ convolution filters to combine the M input channels into N output channels. Applying $1 * 1 * M$ convolutions combines each value in the feature map without considering their spatial relation within the channel.

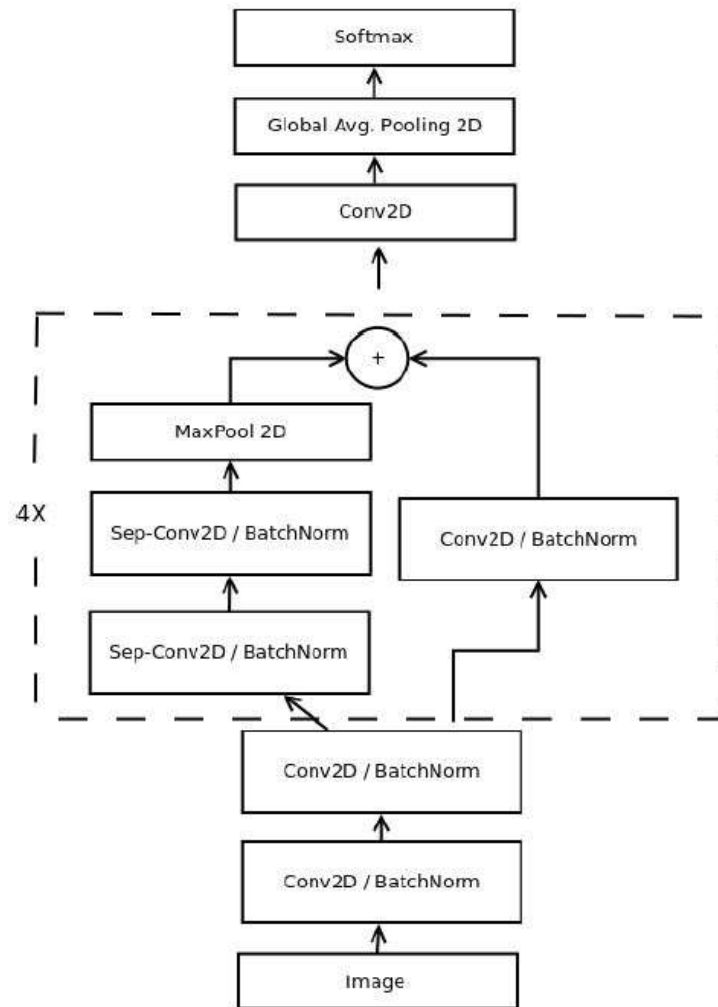


Fig. 3: Our proposed model for real-time classification.

Algorithm(To train CNN Model):-

- 1: $L \times L$ Input Image $\leftarrow$ IMDB Dataset
- 2: Convolution 2D $\leftarrow$ Extract Spatial Features
 - Apply $D \times D$ convolution filter on $L \times L$ with a stride S step size
 - If $S > 1$ downside the image
 - If $S = 1$ size of the image remains same
- 3: Batch Normalization $\leftarrow$ Normalize previous layer at each batch
- 4: if flag ≤ 1

Go to step 2.

Else

Go to step 5.

5: Perform **Depth-wise Separable Convolution**

■ **Perform Depth-Wise Convolution**

$M \leftarrow$ No. of channels $I \leftarrow$ Intermediate Image

Iterate every $D \times D \times M$ kernel to 1 channel of the image to generate $I \times I \times M$ image and stack them

■ **Perform Point-Wise Convolution**

Iterate $1 \times 1 \times M$ kernel to $I \times I \times M$ to get the desired image

■ **Max-Pool 2D** $\leftarrow$ Returns the max value from the position of the image

6: Extraction of residual features from step 3 and store it in a parameter.

7: Add the result obtained after Max-Pool 2D layer and residual feature obtained in previous step.

8: if count ≤ 4

Go to step 5

Else

Go to step 9

9: Apply Convolution 2D over the result of the previous layer (step).

10: Apply **Global Average Pooling 2D** $\leftarrow$ Eliminate fully-connected layers and overfitting

11: Apply **Softmax Function** to predict the output $\leftarrow$ Distinguish between dominant and low-level features

The above algorithm outlines the basic steps to train the CNN Model. It includes the following parameters:

- 1) Convolutional 2D: a “filter” or “kernel” passes over the image, scanning a few pixels at a time and creating a feature map that predicts the class to which each feature belongs.
- 2) Batch Normalization: It normalizes the activation of the previous layer at each batch i.e. applies a transformation that maintains a mean activation close to 0. It achieves the same accuracy with fewer training steps thus speeding up the training process.
- 3) Depth-Wise Convolution: In Depth-Wise Convolution each channel is considered separately and one filter per channel is convolved over that channel only.
- 4) Point-Wise Convolution: From the intermediate result onwards, pointwise convolution mark its beginning. These are filters of 1x1 pixels but which cover all the M intermediate channels generated by the filters.
- 5) Max-Pool 2D: It returns the maximum value from the position of the image covered by the kernel. It acts as a noise suppressant and discards noisy activations altogether.
- 6) Global Average Pooling: Fully connected layers or dense layers have lots of parameters which induces impotency in computational power. The elimination of these layers also reduces the tendency of overfitting.
- 7) Softmax Function: Over a series of epochs, the model is able to distinguish between dominating and certain low level features in images and classify them using the Softmax Function Technique.

Implementation

Pseudocode and Functions

```
def mini_XCEPTION(input_shape, num_classes, l2_regularization=0.01):
    regularization = l2(l2_regularization)
    # base
    img_input = Input(input_shape)
    x = Conv2D(8, (3, 3), strides=(1, 1), kernel_regularizer=regularization,
              use_bias=False)(img_input)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Conv2D(8, (3, 3), strides=(1, 1), kernel_regularizer=regularization,
              use_bias=False)(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    # module 1
    residual = Conv2D(16, (1, 1), strides=(2, 2),
                    padding='same', use_bias=False)(x)
    residual = BatchNormalization()(residual)
    x = SeparableConv2D(16, (3, 3), padding='same',
                      kernel_regularizer=regularization,
                      use_bias=False)(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = SeparableConv2D(16, (3, 3), padding='same',
                      kernel_regularizer=regularization,
                      use_bias=False)(x)
    x = BatchNormalization()(x)
```

```
x = MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)
x = layers.add([x, residual])
```

```
# module 2
```

```
residual = Conv2D(32, (1, 1), strides=(2, 2),
                  padding='same', use_bias=False)(x)
residual = BatchNormalization()(residual)
x = SeparableConv2D(32, (3, 3), padding='same',
                   kernel_regularizer=regularization,
                   use_bias=False)(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = SeparableConv2D(32, (3, 3), padding='same',
                   kernel_regularizer=regularization,
                   use_bias=False)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)
x = layers.add([x, residual])
```

```
# module 3
```

```
residual = Conv2D(64, (1, 1), strides=(2, 2),
                  padding='same', use_bias=False)(x)
residual = BatchNormalization()(residual)
x = SeparableConv2D(64, (3, 3), padding='same',
                   kernel_regularizer=regularization,
                   use_bias=False)(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = SeparableConv2D(64, (3, 3), padding='same',
```

```

        kernel_regularizer=regularization,
        use_bias=False)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)
x = layers.add([x, residual])

# module 4
residual = Conv2D(128, (1, 1), strides=(2, 2),
                  padding='same', use_bias=False)(x)
residual = BatchNormalization()(residual)
x = SeparableConv2D(128, (3, 3), padding='same',
                    kernel_regularizer=regularization,
                    use_bias=False)(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = SeparableConv2D(128, (3, 3), padding='same',
                    kernel_regularizer=regularization,
                    use_bias=False)(x)
x = BatchNormalization()(x)
x = MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)
x = layers.add([x, residual])
x = Conv2D(num_classes, (3, 3),
           # kernel_regularizer=regularization,
           padding='same')(x)

x = GlobalAveragePooling2D()(x)
output = Activation('softmax', name='predictions')(x)
model = Model(img_input, output)

```

return model

The above mentioned is the version of CNN model used in our implementation.

We can see that standard layers are used. Firstly *input* class is used to create an input layer. Then *Conv2D* layer is used to perform the convolution, also we pass the parameters for strides to perform depth-wise convolution. Then we perform *BatchNormalization* and *activation* function. This process is performed two times to make the base for architecture.

Then we move forward towards the modules over base. Here, we find out the residual of the *Conv2D* and *BatchNormalization* layers. Then we apply *SeperableConv2D* layer, over this we perform *BatchNormalization* and *activation* function. We again perform *SeperableConv2D* and *BatchNormalization*. To the output of above layer, we apply *MaxPooling2D*. Output of this layer is added to the residual we got earlier.

This module is repeated 4 times. After the implementation of all the 4 modules, we apply *SoftMax* function to predict the output.

Result

Results of the real-time emotion classification task in unseen faces can be observed in Figure 1(a) & 1(b). Our complete real-time pipeline including: face detection, emotion and gender classification have been fully integrated.

An example of our complete pipeline can be seen in Figure 2(a) & (b) in which we provide emotion and gender classification. These results reassure that the CNN learned to interpret understandable human-like features that provide generalizable elements. These interpretable results have helped us understand several common misclassification such as persons with glasses being classified as “angry”. Moreover, we can also observe that the features learned in our mini-Xception model are more interpretable than the ones learned from our sequential fully-CNN. Consequently the use of more parameters in our naive implementations leads to less robust features.

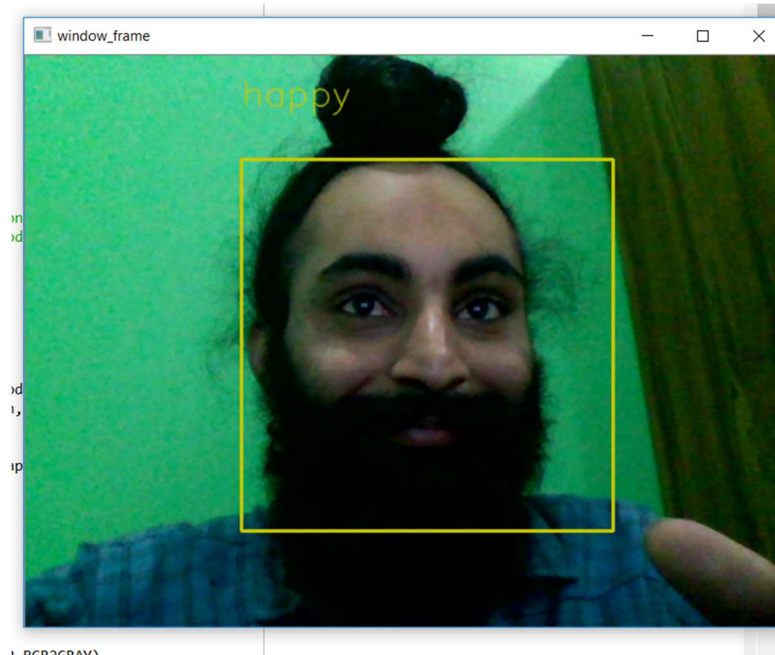


Figure 1 (a)

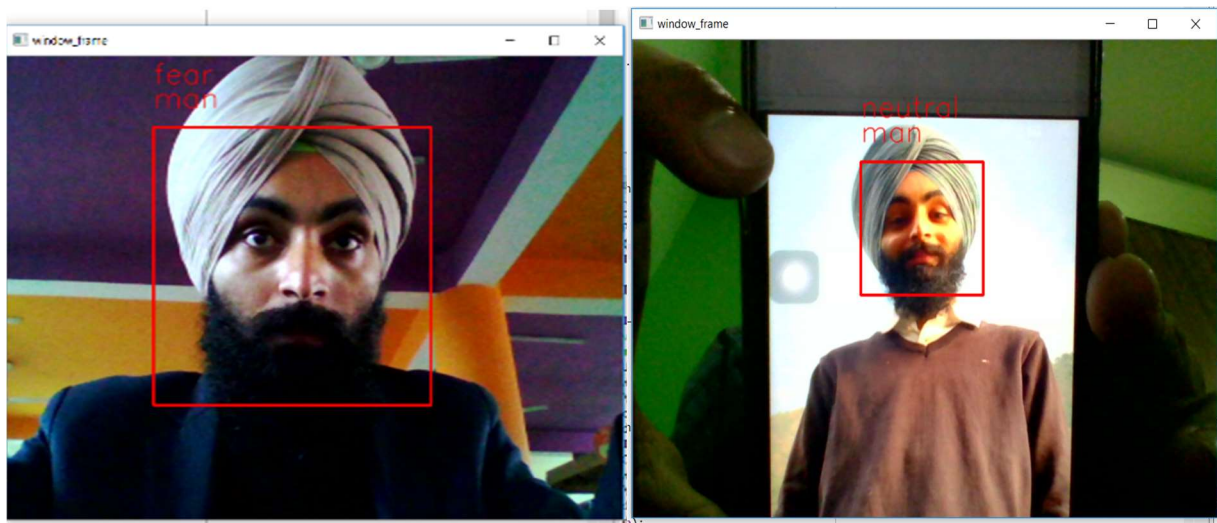


Figure 2 (a)

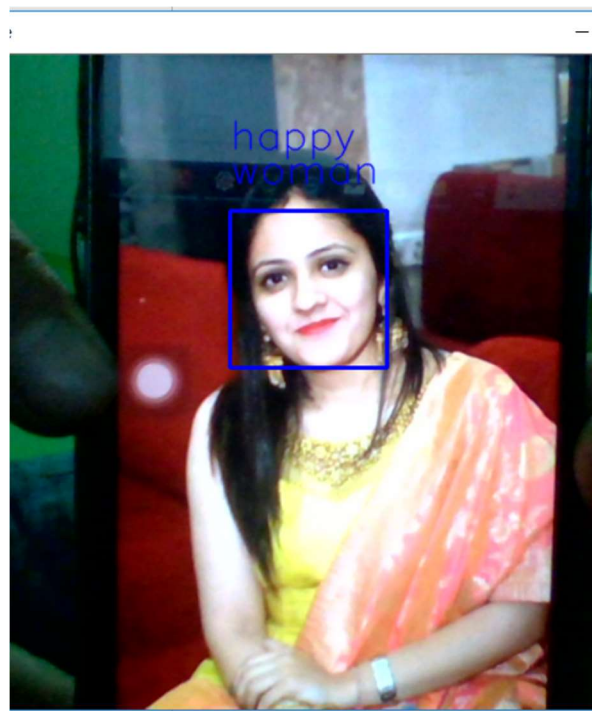


Figure 2 (b)

Conclusion and Future Scope

We have proposed and tested a general building designs for creating real-time CNNs. Our proposed architectures have been systematically built in order to reduce the amount of parameters. We began by eliminating completely the fully connected layers and by reducing the amount of parameters in the remaining convolutional layers via depth-wise separable convolutions. Specifically, we have developed a vision system that performs face detection, gender classification and emotion classification in a single integrated module.

Machine learning models are biased in accordance to their training data. In our specific application we have empirically found that our trained CNNs for gender classification are biased towards western facial features and facial accessories. Furthermore, as discussed previously, the use of glasses might affect the emotion classification by interfering with the features learned. We believe that uncovering such behaviors is of extreme importance when creating robust classifiers, and that the use of the visualization techniques such as guided back-propagation will become invaluable when uncovering model biases.

System Requirements (Software/Hardware)

- Hardware Interface:
 - 32/64 bits processor architecture supported by windows
- Software Interface:
 - Python IDE, Tensor Flow Library